

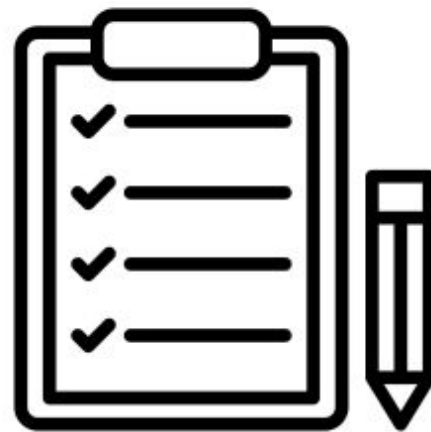


Le génie pour l'industrie

INF111 - Programmation Orienté Objet

Cours #2

École de technologie supérieure - Hiver 2025



Plan de la séance

- **Exemple récapitulatif**
- **Variables et bloc de code**
 - Types
 - Déclaration, utilisation et opération
- **IF/WHILE/FOR**
 - IF-ELSE-IF-ELSE
 - WHILE
 - FOR
- **Normes de programmation**
- **Modules utilitaires**



1. Variables

Il est maintenant venu le temps de discuter des variables.

Tout ce qui est fait dans un ordinateur implique, d'une façon ou d'une autre, **une opération sur la mémoire** (RAM, cache, registre, disque dur et autres I/O). C'est un principe fondamental du fonctionnement des ordinateurs.

La déclaration d'une variable consiste à donner un nom à un espace mémoire. De plus la déclaration de la variable va spécifier quel type d'information sera représenté dans la mémoire.

Variables - types

1. Variables

Voici **les types primitifs** du langage java:

Type	Description	Valeur par défaut	Taille	Exemple Litéral
boolean	vrai ou faux	false	1 bit	true, false
byte	complément à 2	0	8 bits	n/a
char	caractère unicode	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\'', '\n', '\B'
short	complément à 2	0	16 bits	n/a
int	complément à 2	0	32 bits	-2, -1, 0, 1, 2
long	complément à 2	0	64 bits	-2L, -1L, 0L, 1L, 2L
float	IEEE 754 point flottant	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F
double	IEEE 754 point flottant	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d

Source: <http://cs.fit.edu/~ryan/java/language/java-data.html>

Les types les plus utilisés sont le booléen, les caractères, les entiers et les point flottant.

- **booléen (bool)**, utilisé pour stocker un état logique: vrai ou faux
- **char**, utilisé pour stocker une lettre (unicode), ex: 'a', 'z', **'1'**
- **entier (int)**, utilisé pour stocker un entier, ex: -10, 1, 0, 10000
- **point flottant (double)**, utilisé pour stocker un nombre décimal (réel), ex: 10.5, -6.33, 67.34

Variables - étendue

1. Variables

Pour un même code (ex: entier), la taille de l'espace mémoire a un impact sur l'étendu des valeurs qui peuvent être contenue.

Comparer les valeurs possibles pour bytes (entier 8 bits) et long (entier 64 bits)

Type	Taille	Étendu
byte	8 bits	-128 .. 127
short	16 bits	-32,768 .. 32,767
int	32 bits	-2,147,483,648 .. 2,147,483,647
long	64 bits	-9,223,372,036,854,775,808 .. 9,223,372,036,854,775,807
float	32 bits	$3.40282347 \times 10^{38}$, $1.40239846 \times 10^{-45}$
double	64 bits	$1.7976931348623157 \times 10^{308}$, $4.9406564584124654 \times 10^{-324}$

Source: <http://cs.fit.edu/~ryan/java/language/java-data.html>

Variables - déclaration

1. Variables

Voici la syntaxe de **déclaration** d'une variable:

```
<type> nom [= valeur initiale]
```

Le fichier fourni **ExVariables.java** contient une série d'exemples sur comment initialiser des variables de tous types.

Variables - opérations

1. Variables

Il y a toute une série d'opérations qui peuvent être fait sur les variables. Voici quelques syntaxe fondamentales:

```
var = 5; // assigne une valeur littérale  
var += 6; // change la valeur, équivalent à var = var + 6  
var = var2 - var3; // assigne une valeur découlant d'une  
opération sur d'autres variables.
```

Et voici une liste des opérations fondamentales (voir [ExVariables.java](#)):

Type opération	Opérateur
Opérateurs mathématique	+, -, *, /
Opérateurs comparaison	<, >, <=, >=, ==, !=
Opérateurs booléen	&& (AND), (OR)
Raccourci (+/- 1)	var++, var --
Raccourci a = a()b	var += b, var *= b, etc...

Variables - Bloc de code

1. Variables

L'expression bloc de code définit ce qui se trouve entre une ouverture et fermeture d'accolades.

```
{  
    ...Ici, c'est l'intérieur du bloc de code  
}
```

Les blocs de code s'imbriquent les uns dans les autres... Il y a le bloc de code de la classe, puis le bloc de code de la méthode, et plus tard il y en aura d'autre... Il n'est pas rare de voir 4-5 blocs de code imbriqués.

Variables - Bloc de code - portée

1. Variables

La portée d'une variable est une expression qui exprime **la durée de vie**, ou le domaine d'existence d'une variable.

L'existence d'une variable est limité au bloc de code où elle est définie (voir [ExVariables.java](#)).

Questions de Révision

1. Premier programme

- ❑ À quoi sert une variable ?
- ❑ Nommer les 4 types que nous utiliserons le plus et expliquer le type de données qu'elles peuvent contenir.
- ❑ Expliquer la relation entre bloc de code et portée d'une variable.

Réponses

1. Premier programme

- ❑ Donner un nom à un espace mémoire
- ❑ bool, char, int, double
valeur true/false, caractère (symbole), entier, réel
- ❑ Une variable existe pour tout l'étendu du bloc de code dans lequel elle a été définie.

Exemples en classe

1. Premier programme

Écrire un petit programme qui demande 2 entiers à l'utilisateur et affiche la somme.

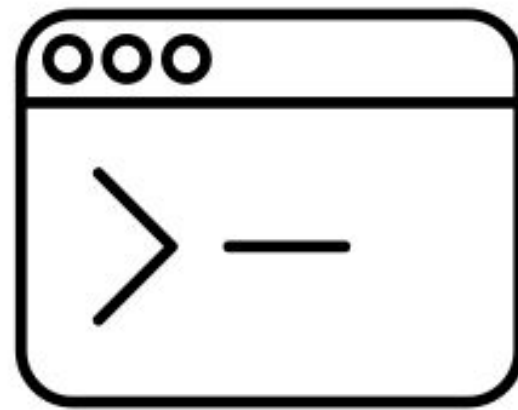
Pour les besoins de l'exercice, le résultat de la somme des deux variables devra d'abord avoir été stocké dans une autre variable.

Solution dans `ExVariables::Exemple1.java`

Ouverture de sujet

1. Premier programme

❑ Opération sur la mémoire



2. Contrôle de flow

En informatique, les termes *expression* ou *condition* sont utilisées dans la documentation pour indiquer que l'on peut insérer un petit bout de code qui sera résolue.

Spécifiquement, une *condition* indique que ce qui est inséré doit se résoudre comme une opération booléenne, c'est à dire que le résultat sera true/false.

Voici des exemples de conditions:

(condition)

- **(a==5)**, cette condition retourne *vrai*, si le contenu de la variable *a* est égal à 5.
- **(a<=b)**, cette condition retourne *vrai*, si le contenu de la variable *a* est plus petit ou égal au contenu de *b*.
- **(a<=b && a>=c)**, cette condition retourne *vrai*, si le contenu de la variable *a* est plus petit ou égal au contenu de *b* et plus grand ou égal au contenu de *c*.

Jusqu'à maintenant, tous les exemples de code écrit sont limité à s'exécuter de haut en bas, et chaque ligne de code est exécutée inconditionnellement.

Cela limite grandement la flexibilité de nos programmes et la complexité des problèmes pouvant être résolues.

Explorons les outils de contrôles de flow qui permettent d'avoir plus de contrôle sur l'exécution du code. Nous en verrons trois:

- **IF**
- **While**
- **For**

Contrôle du flow - IF, ELSE-IF, ELSE

2. Contrôle de flow

Le IF se lit comme SI. Il s'agit d'une structure de contrôle qui **permet d'appliquer une condition à l'exécution d'un bloc de code**. Dans sa forme générale, on peut lire:

SI la condition est vraie, exécute ce bloc de code.

```
if (condition) {  
    //bloc de code exécuté conditionnement  
}
```

(voir ExIf.java)

Contrôle du flow - IF, ELSE-IF, ELSE

2. Contrôle de flow

Au delà du IF de base, il est possible d'ajouter des structures au IF, pour aller chercher d'autres modes d'opération.

IF-ELSE	IF-ELSEIF
<pre>if(cond1) { // exécuté si cond1 est vraie }else{ // exécuté si cond1 est fausse }</pre>	<pre>if(cond1) { // exécuté si cond1 est vraie }else if(cond2) { // exécuté si cond1 est fausse et // cond2 est vrai }</pre>
IF-ELSEIF-ELSE	
<pre>if(cond1) { // exécuté si cond1->vraie }else if(cond2) { // exécuté si cond1 est fausse et // cond2 est vrai }else{ // exécuté si cond1 et cond2 sont // fausses }</pre>	

- De manière générale, il n'y a pas de limite pratique au nombre de ELSE-IF qui peuvent être ajouté à la suite du IF.
- Il ne peut y avoir qu'un seul ELSE, et il est optionnel.
- Le if évalue une condition, les conditions les plus fréquentes prend la forme suivante:

(varA op. comparaison varB)

Par exemple:

`(varA == varB) ,`

`(varA < varB)`

`(varA >= varB)`

`(varA != varB)`

Contrôle du flow - IF, ELSE-IF, ELSE

2. Contrôle de flow

Il y a aussi possibilité de combiner des conditions simples avec les opérateurs booléens:

(varA op. comparaison varB) op. booléen (varC op. comparaison varD)

Par exemple:

```
( (varA < varB) && (varA > varC) ),  
( (varA == varB) || (varA == varC) )
```

Contrôle du flow - WHILE

2. Contrôle de flow

Le while, se dit TANT QUE ou JUSQU'À CE QUE. Cette structure de contrôle permet de répéter le bloc de code, tant que la condition fournie est vraie.

Voici la syntaxe de base du while:

```
while (cond1) {  
    // bloc de code qui se répète  
}
```

Pour le while, la condition suit la même ligne directrice que pour le IF. Faites attention, d'ailleurs, certains étudiants confondent le IF et le WHILE, alors que les deux sont complètement différents.

(voir [ExBoucle.java](#))

La boucle FOR se dit POUR TOUS LES. C'est un cas particulier de la boucle WHILE. La boucle FOR implémente un compte inconditionnel et est très fréquemment utilisé avec les tableaux que nous verrons la semaine prochaine.

Voici la correspondance entre la boucle while et la boucle FOR.

Boucle while	FOR
<pre>int compte = 0; while(compte < LIMITE){ // code exécuté par la boucle compte++; }</pre>	<pre>int compte = 0; for(int compte = 0;compte<LIMITE;compte++){ // code exécuté par la boucle }</pre>

La boucle FOR peut être écrite pour compter en montant, en descendant, par saut de 1, -1 ou n'importe quel autre chiffre. Très souvent, on utilise i, j, ou k, comme nom d'itérateurs.

```
for (int i=0; i<10; i++) {  
    //code à répéter  
}
```

```
for (int i=10; i>0; i--) {  
    //code à répéter  
}
```

```
for (int i=0; i<100; i+=5) {  
    //code à répéter  
}
```

Contrôle du flow - FOR vs. WHILE

2. Contrôle de flow

Pour savoir quand utiliser la boucle FOR ou la boucle WHILE, une question suffit.

Est-ce que le nombre d'itération de la boucle est connue au moment de lancer la boucle?

Si la réponse est oui, alors c'est la boucle FOR, sinon c'est la boucle while.

Questions de Révision

2. Contrôle de flow

- ❑ Est-ce qu'il y a des situations où on a le choix d'utiliser un if ou un while ?
- ❑ Est-ce que les deux exemples suivants sont équivalents ?

```
if(var == 20) {  
    printf("A\n");  
}  
else if(var > 10) {  
    printf("B\n");  
}
```

```
if(var == 20) {  
    printf("A\n");  
}  
  
if(var > 10) {  
    printf("B\n");  
}
```

Questions de Révision

2. Contrôle de flow

- ❑ La séquence `if → else → else if → else` est valide, puisque chaque `else` est précédé d'un `if`.
- ❑ Est-ce que les deux exemples suivants sont équivalents ?

```
while (i < 20) {  
  
    // code...  
  
}
```

```
while( !(i >= 20) ){  
  
    // code...  
  
}
```

- ❑ Pour chacune des situations suivantes indiquer si l'on doit utiliser un WHILE ou un FOR:
 - Demander son mot de passe à l'utilisateur, jusqu'à ce qu'il soit bon ?
 - Simuler le lancement d'un dé 1000 fois.
 - Demander son mot de passe à l'utilisateur, jusqu'à ce qu'il soit bon, mais avec un maximum de 3 essais
 - Demander à l'utilisateur, combien de registres doivent être rempli, puis lui demander de remplir chaque registre un à un.

Réponses

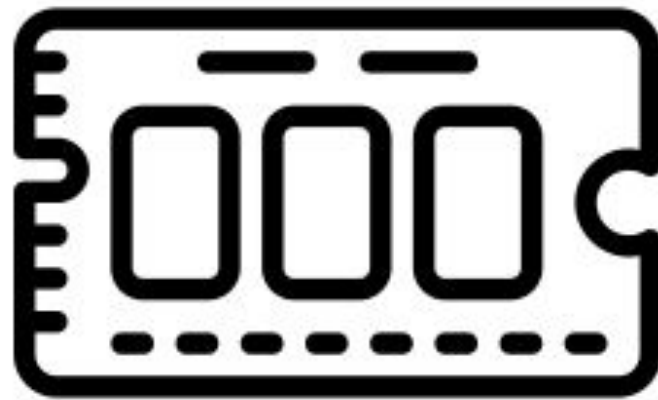
2. Contrôle de flow

- ❑ Absolument pas, if et while sont complètement différent.
- ❑ Non, dans le premier cas, seul 'A' est affiché. Dans le second cas, 'A' et 'B' s'affichent.
- ❑ Non, ne peut avoir qu'un seul else et else ne peut pas être suivi par une autre condition
- ❑ Oui.
- ❑ WHILE, FOR, WHILE, FOR

Écrire un programme simulant le jeu du nombre caché.

- **Générer un nombre aléatoire entre 1 et 20, inclusivement.**
- **Demander à l'utilisateur de deviner le nombre.**
- **À chaque tentative, indiquer si le nombre caché est plus petit ou plus grand.**
- **Quand l'utilisateur trouve le nombre, afficher bravo! et indiquer le nombre de tentatives qui ont été requises.**

- ❑ Il existe un bug que l'on appelle une boucle infinie. Pouvez-vous spéculer sur ce en quoi ça consiste ? Quelle serait votre stratégie pour déboguer ce code ?



3. Normes de programmation

Dans tous les contextes de programmation, il y a un certain nombre de règles à suivre qui ne sont pas des règles syntaxiques, mais plutôt des règles de bienséance. Ce sont les normes de programmation.

Les normes aident notamment à améliorer et faciliter le contrôle de la qualité dans le développement logiciel et uniformiser le code produit au sein d'une organisation.

Dans le cadre du cours, vous avez à suivre un certain nombre de ces règles.

Nom des variables et des méthodes:

- **camelCase**, première lettre minuscule, première lettre des mots qui suivent Majuscule.

Nom des constantes (semaine prochaine):

- **TOUT_EN_MAJUSCULE**, avec des '_' pour espacer les mots

Nom des classes (semaine 3):

- **ExempleDeNomDeClasse**, première lettre majuscule, de même que les début de mots qui suivent.

Normes de programmation - en-tête de fichiers

3. Normes de programmation

Tous les fichiers et les méthodes doivent utiliser la syntaxe javadoc.

Canvas d'en-tête de fichier:

```
/**  
 * La description de la classe s'écrit ici  
 *  
 * @author (le nom de tous les auteurs)  
 * @version (référence temporelle ex: Hiver 2018)  
 */
```

Normes de programmation - en-tête de méthodes

3. Normes de programmation

Canvas d'en-tête de méthode:

```
/**  
 * La description de la méthode est à écrire ici  
 *  
 * @param nom, description  
 * @param nom, description  
 * @return descripton  
 */
```

Normes de programmation - en-tête de fichiers et méthodes

3. Normes de programmation

Cette page contient plusieurs exemples d'en têtes de fichiers et méthodes:

https://www.tutorialspoint.com/java/java_documentation.htm

Dans le cadre du cours, vous pouvez vous limiter à l'exemple dans [Canvas.java](#).

Votre code doit également contenir des commentaires en ligne.

- **Pour les court commentaires utiliser les ‘//’.**
- **Pour les commentaires long, utiliser les ‘/*…* /’**

Les commentaires doivent être au même niveau que le code par exemple:

```
// Si le mot de passe est valide, notifie le systeme que
// l'utilisateur a été autentifier, tel que spécifier au
// requis 5.6
if (motDePasseValide (motDePasse) ) {
    autoriseAuthentification();
}
```

Indentation, l'indentation fait référence à la tabulation insérer au début de la ligne. Le nombre de tabulation doit correspondre au nombre de bloc de code de profondeur.

aération, n'essayez pas de réduire le nombre de ligne de code en enlevant les lignes vides. La façon de faire est de laisser une ligne vide entre chaque section logique.

Code inutile et/ou commenté, en cours de développement, vous accumulez des lignes de codes en commentaires ou désuètes. C'est normal, par contre, assurez-vous de les enlever avant la remise.

Questions de Révision

2. Contrôle de flow

- ❑ À quoi servent les normes ?
- ❑ Nommer 3 normes à considérer, parmi toutes celles demandées ?

Réponses

2. Contrôle de flow

- ❑ Uniformiser le code, réduire les risques d'erreurs
- ❑ format d'écriture des variables/fonctions, indentation/aération, entêtes, commentaires

Ouverture de sujet

2. Contrôle de flow

- ❑ **Peut-on perdre son emploi pour une question de normes, si notre code fonctionne?**
- ❑ **Quel est la meilleure manière de mettre les normes en place (en 2025)?**



4. Modules utilitaires

L'environnement Java vient avec une immense collection de modules utilitaires et classes qui contiennent des outils de programmation pour aider à la conception et accélérer la production de code.

Il y en a beaucoup, on en couvre plusieurs au cours de la session, mais ce ne serait pas possible de tous les découvrir. Développer sa connaissance des modules disponibles fait partie de l'apprentissage du langage Java.

Aujourd'hui, on en étudie deux, ce qui nous permettra de comprendre les principes de base de l'utilisation des modules.

Les modules de la journée: Math et Random

Modules utilitaires - import

4. Modules utilitaires

La commande import permet d'indiquer au compilateur que l'on désire avoir accès aux fonctionnalités d'un module.

Par exemple, si je veux avoir accès à la classe Random, je peux écrire:

```
import java.util.Random;
```

Dans l'import, ce que l'on indique c'est le *path* de la classe que l'on veut inclure.

Dans le case de *java.util.Random*

- *java* est le path de base pour les modules (libraries en anglais) du kit de développement java (JDK),
- *util* est un sous ensemble, qui contient, entre autres, la classe *Random*.

Il arrive parfois qu'on ait de besoin de plusieurs classes dans un path donnée, on peut alors écrire:

```
import
```

```
java.util.*;
```

Ce qui importe toutes les classes.

Le module Math offre une série de méthode couvrant les fonctionnalités mathématiques de base, révisons ensemble la documentation du module:

<https://docs.oracle.com/javase/8/docs/api/java/lang/Math.html>

- Le module Math est dans `java.lang.*`
- Une particularité du module Math est qu'il est statique. On n'a donc pas de besoin de créer d'instance pour l'utiliser.
- On peut d'ailleurs faire un `import static`, c'est le seul module que je connais pour lequel c'est le cas.
- Quand on fait un `static import`, on n'a plus à écrire `Math.abs(...)`, mais juste `abs(...)`.

Voir `ExModules.java`, pour quelques exemples de l'utilisation de Math.

Le module Random offre une série de méthode couvrant les fonctionnalités d'un générateur pseudo-aléatoire de base, révisons ensemble la documentation du module:

<https://docs.oracle.com/javase/7/docs/api/java/util/Random.html>

- Le module Math est dans java.util.*
- La classe Random n'est pas statique, il faut donc l'instancier pour l'utiliser.

Voir ExModules.java, pour quelques exemples de l'utilisation de Random.

Focus sur: `nextInt()`, `nextInt(int n)`

4. Modules utilitaires

.nextInt est une méthode qui permet d'obtenir un entier distribué uniformément. La méthode s'appelle de deux manières:

- `nextInt()`, un entier aléatoire entre 0 et $2^{32}-1$
- `nextInt(int n)`, un entier aléatoire entre 0 et $n-1$

Ex:

```
// un entier entre 0 et 9 (10 est exclus)  
int monEntier = rand.nextInt(10);
```

Dans le cadre du cours, on l'utilise pour : simuler un dé, ajouter un peu d'aléa dans une simulation, tirer un nombre caché, etc...

Focus sur: Le modulo

4. Modules utilitaires

Le **modulo** est l'opérateur qui **retourne le reste d'une division entière**. Par exemple, si on divise 5 par 2, alors le résultat de la division entière est 2 (car 2×2 entre dans 5, mais pas 3×2) et le reste est 1.

Le modulo utilise l'opérateur %

Voici l'exemple décrit plus haut en code:

```
int restant = 5%2;  
// restant est maintenant égale à 1
```

Focus sur: Le modulo

4. Modules utilitaires

Ce qui est particulier dans le cas du modulo, c'est que cette opération a un paquet d'applications. Par exemple, on peut l'utiliser pour tester si un nombre est pair ou impair.

```
var % 2 == 1 // vrai si var est impair
```

On peut aussi l'utiliser pour faire un compteur circulaire

```
// ce code affiche en boucle  
0,1,2,3,4,0,1,2,3,4,0,1,2,3,4  
int var = 0  
for(int i=0;i<100;i++){  
    var += 1;  
    var %= 5  
    System.out.println(var)  
}
```

Questions de Révision

2. Contrôle de flow

- ❑ Décrire le fonctionnement du module
- ❑ Quels sont les valeurs possibles de `var % 5`
- ❑ Comment tirer un nombre aléatoire entre
 - `[50,100]`
 - `[50,100[`
- ❑ Comment je m'y prend pour importer la classe `PriorityQueue`

Réponses

2. Contrôle de flow

- ❑ Retourne le reste de la division entière
- ❑ 0 à 4
 - ❑ `rand.nextInt(50) + 50`
 - ❑ `rand.nextInt(50+1) + 50`
- ❑ Identifie le path dans la documentation, puis écrit le import:
`import java.util.PriorityQueue`

Exercice Récapitulatif

2. Contrôle de flow

Écrivez un code qui tire un nombre aléatoire entre 0 et 100 et le met dans une variable. Il demande ensuite à l'utilisateur d'entrer un nombre. Le but de l'utilisateur est de trouver le nombre qui a été tiré par l'ordinateur.

Le programme s'exécute TANT QUE le nombre entré par l'utilisateur est différent du nombre tiré par l'ordinateur.

À chaque essai, l'ordinateur indique à l'utilisateur SI le nombre entré est trop petit ou SI il est trop grand. Puis redemande un nombre.

Résumé: Exemple 1

Points à remarquer

- Utilisation du module Random pour obtenir un nombre aléatoire.
- Utilisation de la boucle pour répéter une opération, tant que le résultat permettant de continuer n'est pas obtenu.
- Utilisation du if, pour choisir le message à afficher.